

CS 425 : Computer Networks

Homework-1

Name : P.Sruthi

RollNo :-230751

2. In Go-Back-N ARQ with k-bit sequence numbers, the sequence number space is 0 to $(2^k)-1$ (total 2^k sequence numbers). If the sender window size is 2^k , sequence numbers would wrap around, and a retransmitted old frame could have the same sequence number as a new frame, so the receiver cannot distinguish duplicate frames from new ones, causing ambiguity and unreliable delivery.

Therefore, the maximum sender window size is limited to 2^k-1 so that at least one sequence number remains unused, allowing the receiver to uniquely identify new and duplicate frames and ensure reliable communication.

3. In Selective-Reject ARQ, where sequence numbers are represented using k bits, so the total sequence number space is : 0 to $(2^k)-1$ (total 2^k sequence numbers).Both the sender window and receiver window can hold multiple frames simultaneously, because the receiver accepts and buffers out-of-order frames. After sequence number wrap-around, the new window positions may overlap with old sequence numbers still stored in the receiver window if the window size is too large.

To ensure that old and new windows never overlap, the window size must be at most half of the total sequence number space : Maximum window size = $2^{(k-1)}$

This guarantees:

Clear separation between old and new frames in the sliding window

Unambiguous frame identification

4. For Stop-and-Wait ARQ, efficiency is given by

$$U = \frac{T_{frame}}{T_{frame} + 2T_p}$$

where:

- $T_{frame} = \frac{L}{R}$ is the frame transmission time
- L = frame size (bits)
- R = data rate
- T_p = one-way propagation delay

Given $R = 4 \text{ kbps} = 4000 \text{ bps}$, $T_p = 20 \text{ ms} = 0.02 \text{ s}$

$$T_{frame} = \frac{L}{4000}$$

Condition for at least 50% efficiency

- $U \geq 0.5$

$$\frac{L/4000}{L/4000 + 2(0.02)} \geq 0.5$$

$$\frac{L/4000}{L/4000 + 0.04} \geq 0.5$$

$$\frac{L}{4000} \geq 0.5 \left(\frac{L}{4000} + 0.04 \right)$$

$$L \geq 0.5(L + 160)$$

$$0.5L \geq 80$$

$$L \geq 160 \text{ bits}$$

Stop-and-Wait ARQ achieves efficiency $\geq 50\%$ when the frame size is at least 160 bits.
Hence, the required range of frame sizes is:

$$L \geq 160 \text{ bits}$$

5. Considering a frame consisting of one character of 4 bits and assuming that the probability of bit error is let $p = 10^{-3}$ and that it is independent in each bit :

(a) Probability that the *received* frame contains no errors

Probability that a bit is received correctly:

$$1 - p = 1 - 10^{-3} = 0.999$$

Since the frame has 4 independent bits, the probability of no error in the frame is:

$$\begin{aligned} P(\text{no error}) &= (0.999)^4 \\ P(\text{no error}) &\approx 0.996006 \end{aligned}$$

(b) Probability that the received frame contains at least one error

Using the complement rule:

$$\begin{aligned} P(\text{at least one error}) &= 1 - P(\text{no error}) \\ &= 1 - 0.996006 \\ &\approx 0.003994 \end{aligned}$$

(c) Probability of undetected error when one parity bit is added

With one parity bit, total bits in the frame = 5.

Parity detects odd number of bit errors and fails to detect even number of bit errors. Thus,

undetected error occurs when there are 2 or 4 bit errors.

Using the binomial distribution:

$$P(\text{undetected}) = \binom{5}{2} p^2 (1-p)^3 + \binom{5}{4} p^4 (1-p)$$

Substituting $p = 10^{-3}$: $10 \times (10^{-6}) \times (0.999)^3 + 5 \times (10^{-12}) \times (0.999)$
 $\approx 9.97 \times 10^{-6}$

$P(\text{no error}) \approx 0.996006$

$P(\text{at least one error}) \approx 0.003994$

$P(\text{undetected error with parity}) \approx 9.97 \times 10^{-6}$

6 . Given

Generator polynomial :- $P = 110011$ (degree = 5)

Message : $M = 11100011$

To find CRC :- Append 5 zeros (degree of generator) : Dividend = 11100011 00000

```

      10110110
-
110011 ) 1110001100000
.      110011
.      -----
.      00101111
.      110011
.      -----
.      0111000
.      110011
.      -----
.      00101100
.      110011
.      -----
.      0111110
.      110011
.      -----
.      0011010 → remainder

```

Message : 11100011

CRC : 11010

Frame : 1110001111010

7.

(a) Given Generator polynomial : $P(x) = X^4 + X + 1$

Binary generator : 10011

Message : 10010011011

Generator degree = 4 $\rightarrow$ append 4 zeros : Dividend = 10010011011 0000

```

      10001010
      -----
- 10011 ) 100100110110000
      10011
      -----
      000010110
      10011
      -----
      0010111
      10011
      -----
      0010000
      10011
      -----
      0001100  $\rightarrow$  CRC

```

Final Encoded Frame : Codeword = Message + CRC
= 10010011011 1100
100100110111100

(b) **Received frame** : 000110110111100

Divide by 10011 : Remainder = 1110 $\neq$ 0000

$\Rightarrow$ Error detected

(c) **Received frame** : 000010110111100

Division remainder : 0000

$\Rightarrow$ Error NOT detected

Reason:

The error polynomial is **divisible by the generator**, so CRC cannot detect it.

1.

Part(2) :- Just run the code attached to this report. The results will be printed out in the terminal .

```
===== CRC ENCODING TEST CASES =====

Test Case 1
Data: 1011110111
CRC: 10111
Transmitted Frame: 101111011110111

Test Case 2
Data: 0000111000
CRC: 01000
Transmitted Frame: 000011100001000

===== PART-2: ERROR DETECTION =====

Original 10-bit Data: 0010010111
Generated CRC: 00100
15-bit Transmitted Frame T: 001001011100100

Received Frame (after errors): 000001001100100
Result: Frame DISCARDED (Error detected)
```

Part(1) :

```
===== CRC FRAME GENERATION =====

Enter k-bit data block D: 1011110111
Enter CRC generator pattern P: 10011

CRC remainder: 0010
n-bit transmitted frame: 10111101110010
```

```

Welcome  hi.cpp X
hi.cpp
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5
6  /* XOR operation used during CRC division */
7  string xor_op(string a, string b) {
8      string result = "";
9      for (size_t i = 1; i < b.size(); i++) {
10         result += (a[i] == b[i]) ? '0' : '1';
11     }
12     return result;
13 }
14
15
16 /* Perform Modulo-2 division and return CRC remainder */
17 string mod2div(string dividend, string divisor) {
18     int pick = divisor.length();
19     string tmp = dividend.substr(0, pick);
20
21     while (pick < dividend.length()) {
22         if (tmp[0] == '1')
23             tmp = xor_op(divisor, tmp) + dividend[pick];
24         else
25             tmp = xor_op(string(pick, '0'), tmp) + dividend[pick];
26         pick++;
27     }
28
29     if (tmp[0] == '1')
30         tmp = xor_op(divisor, tmp);
31     else
32         tmp = xor_op(string(pick, '0'), tmp);
33
34     return tmp;    // CRC remainder
35 }
36
37
38 /* Generate transmitted n-bit frame */
39 string generate_frame(string data, string key, string &crc) {
40     int r = key.length() - 1;
41

```

This program implements the **Cyclic Redundancy Check (CRC)** technique to generate an **n-bit transmission frame** from a **k-bit data block** using a given **CRC generator pattern**.

The user provides the input data and generator polynomial, after which the program appends zeros equal to the degree of the polynomial and performs **modulo-2 division using XOR operations** to compute the CRC remainder.

The calculated remainder is then appended to the original data to form the **final transmitted frame**.

The program can be executed with **different input values** to verify that the CRC generation process works correctly for multiple test cases.

```
Welcome  hi.cpp  X
G hi.cpp
38  /* Generate transmitted n-bit frame */
39  string generate_frame(string data, string key, string &crc) {
40      int r = key.length() - 1;
41
42      // Append r zeros
43      string appended_data = data + string(r, '0');
44
45      // Compute CRC
46      crc = mod2div(appended_data, key);
47
48      // Final transmitted frame
49      return data + crc;
50  }
51
52
53  /* ===== MAIN ===== */
54  int main() {
55
56      cout << "==== CRC FRAME GENERATION =====\n\n";
57
58      string data, key;
59
60      // Take user input
61      cout << "Enter k-bit data block D: ";
62      cin >> data;
63
64      cout << "Enter CRC generator pattern P: ";
65      cin >> key;
66
67      string crc;
68      string frame = generate_frame(data, key, crc);
69
70      // Display results
71      cout << "\nCRC remainder: " << crc << endl;
72      cout << "n-bit transmitted frame: " << frame << endl;
73
74      return 0;
75  }
76  |
```